

QA Technical Assessment

Quality Engineering — Mid-Senior / Senior / Lead

Before You Start

Thank you for your interest in joining our Quality Engineering team.

This assessment is **not a test of how fast you can write code**. We are interested in how you think: how you read a requirement, where you look for risk, what you choose to test, what you choose *not* to test, and how you would design something that another engineer can maintain after you.

Please read the rules below before you begin.

Time and scope

The per-task times below cover the thinking and the writing. Allow **another 20–30 minutes** on top for setting up your repository, capturing evidence, exporting your Postman files and writing your README.

Level	Expected total effort, including packaging
Mid-Senior	Around 2 hours
Senior	Around 2 to 2.5 hours
Lead	Around 2.5 to 3 hours

Deadline: 2 working days from the day you receive this pack. If that doesn't work for you, reply and ask — it is not held against you.

If you run out of time, submit what you have and write down what you would have done next. An honest, well-reasoned partial answer scores higher than a padded complete one.

Which tasks apply to you

Task	Time	Mid-Senior	Senior	Lead
Task 1 — Test Design & Risk Coverage	40 min	Required	Required	Required
Task 2 — Exploratory Testing & Bug Reporting	25 min	Required	Required	Required
Task 3 — API	30 min	Required	Required	Required

Task	Time	Mid-Senior	Senior	Lead
Testing				
Task 4 — Automation Design (no code)	35 min	Required	Required	Required
Task 5 — Quality Strategy & Release Judgement	30 min	—	Optional (bonus)	Required

How to submit

1. Put your work in a **Git repository** (GitHub / GitLab / Bitbucket — public, or private with access shared to the address in your invitation email) and send us the link. A single ZIP is accepted only if a repository is not possible for you.
2. Use a clear folder per task: task-1/, task-2/, task-3/, task-4/, task-5/.
3. Include a top-level README.md containing:
 - **Time spent on each task**, honestly recorded.
 - **Your assumptions and the questions you would have asked** a Business Analyst or Product Owner if you could.
 - Anything you deliberately left out and why.
4. Documents may be Markdown, PDF, Word or Excel — whatever you are most comfortable with. Content matters, formatting does not.

Using AI tools

You may use AI tools. We use them ourselves.

If you do, include an ai-usage.md file with the prompts you used and a short note on **what you kept, what you rejected, and why**. Using AI well is a skill we value.

You will be asked to walk through your submission in detail during the technical interview — the decisions you made, the alternatives you considered, and what you would change. That conversation is where the real assessment happens, so make sure the work is something you can stand behind and explain.

One thing to keep in mind

Some of the requirements below are **incomplete or ambiguous on purpose**. That is not an oversight. Spotting the gap and asking the right question is a large part of what we are assessing.

Task 1 — Test Design & Risk Coverage

Suggested time: 40 minutes

Context

You have joined a project delivering the mobile banking app for a retail bank. The team is about to build the **Local Fund Transfer** feature. The following is the entire requirement the team has been given.

Requirement (as received from the Business Analyst)

As a retail customer, **I want to** transfer money from my account to a beneficiary I have already registered, **so that** I can send funds without visiting a branch.

AC1 — Transfer initiation

The customer selects a source account from their list of accounts, selects a beneficiary from their registered beneficiary list, enters a transfer amount, and may optionally enter a purpose note.

AC2 — Authentication

The transfer must be confirmed with a One-Time Password before it is executed.

AC3 — Fees

A transfer fee is applied according to the bank's fee configuration.

AC4 — Limits

Transfers are subject to a daily transfer limit of 20,000 and a per-transaction limit of 10,000.

AC5 — Successful transfer

On success the source account is debited, a transaction reference number is generated, a confirmation screen is displayed with the transfer details, and a confirmation SMS is sent to the customer.

AC6 — Failure

If the transfer cannot be completed, the customer is shown an error message.

What we would like from you

1.1 — Questions and assumptions List the questions you would raise with the Business Analyst, Product Owner or the bank's business team before testing begins. For each one, state briefly *why it matters*. Where you cannot get an answer, state the assumption you would test against.

Guidance: quality beats quantity. A focused list of sharp questions is worth more than thirty generic ones.

1.2 — Risk assessment Identify the areas of this feature you consider highest risk and explain your reasoning. Say what could go wrong, and what the impact would be on the customer, on the bank, and on us as the delivery partner.

1.3 — Test cases Write your test cases for this feature — **maximum 12**. Use whatever template you normally use, but each case should make clear what is being tested, the preconditions, and the expected result.

The 12-case cap is deliberate. We want to see what you consider worth testing when you cannot test everything.

1.4 — Coverage decisions Briefly answer:

- What did you deliberately choose **not** to cover, and why?
- Which of your cases belong in an automated regression suite, and which stay manual? Why?
- How would you know this feature is safe to release?

Task 2 — Exploratory Testing & Bug Reporting

Suggested time: 25 minutes

Context

Use the public demo banking application below. It is an intentionally imperfect demo application.

Primary site: <https://parabank.parasoft.com/parabank/index.htm> (*Register a new user, then explore: Accounts Overview, Open New Account, Transfer Funds, Bill Pay, Find Transactions, Request Loan, Update Contact Info.*)

Backup site, if the primary is unavailable: <https://demo.testfire.net/>

If neither site is reachable on the day you attempt this, say so in your README and use any live banking or payments demo application you can access — tell us which one you used. This will not count against you.

How this task is scored

We are not checking whether you found particular defects. These are public demo sites; their behaviour and data change over time, and we know that. You are scored on:

- Your exploration approach and where you chose to look
- Whether someone else could reproduce your finding from your report alone
- The quality of your evidence
- Your actual-versus-expected reasoning, and where your expectation comes from
- Whether your severity and priority are justified in business terms

A candidate who reports two defects precisely will score higher than one who reports five vaguely.

What we would like from you

2.1 Report 2 functional bugs. For each one provide:

- A title that tells a reader what is broken without opening the report
- Environment and preconditions
- Steps to reproduce, numbered and reproducible by someone who has never seen the app
- Actual result vs expected result — and state *where* your expectation comes from
- Severity and Priority, **with a one-line business justification for each**
- Evidence (screenshot, response body, console/network detail — whatever supports the finding)

2.2 In no more than half a page: describe the approach you took. What did you look at first, and why? If you had two more hours on this application, where would you look next?

Notes

- Functional defects only. Cosmetic and UI-polish observations do not count.
- No duplicates — two distinct root causes, not two symptoms of the same bug.
- Do not perform any security, load, or penetration testing on these sites.

Task 3 — API Testing

Suggested time: 30 minutes

This task has two parts. Part A is hands-on. Part B is written analysis — no code required.

Part A — Build and run a small collection

A banking app shows customers an indicative FX rate before they send an international transfer. The rate comes from an upstream rates service.

Use this free, no-key public API:

`https://open.er-api.com/v6/latest/USD`

(Backup if it is unavailable: `https://api.frankfurter.app/latest?from=USD`)

Use whichever one works for you and tell us which you used. The two providers return different shapes, error formats and currency sets. You are assessed on how well you understand and validate **the API you actually used** — not on producing one particular expected response code.

Build a **Postman collection** that:

1. Retrieves rates for a base currency and validates the response — status, response time, schema/shape, and that the expected target currencies are present with sensible values.
2. Runs the **same requests across multiple base currencies driven by an external data file** (CSV or JSON), not by duplicating requests.
3. Uses **variables and environments** rather than hard-coded values.
4. **Captures at least one response value and uses it meaningfully** in a subsequent request or validation. Briefly explain why you chose that value.
5. Includes at least one **negative case** — an unsupported or malformed currency code. Assert on how the API you chose actually behaves, and note whether that behaviour is what you would want from a rates provider in a banking app.

Submit: the exported collection, the environment file, the data file, and a short note on what your assertions verify and why you chose those.

Part B — Testing a payments endpoint (written, no code)

The team is exposing this endpoint for outgoing transfers:

POST /api/v1/payments

Headers: Authorization, Idempotency-Key, X-Channel

Body:

```
{
  "sourceAccountId": "string",
  "beneficiaryId": "string",
  "amount": { "value": 1500.75, "currency": "SAR" },
  "fxQuoteId": "string, optional",
```

```
"purposeCode": "string",  
"note": "string, optional"  
}
```

Responses:

```
201 Created -> { paymentId, status: "PENDING|COMPLETED", reference,  
fee, valueDate }  
400 Bad Request  
402 Insufficient Funds  
409 Conflict  
422 Limit Exceeded  
503 Service Unavailable
```

Answer these three. Bullets are fine.

3.1 What are the highest-risk scenarios you would test on this endpoint? Focus on what makes this a *money* endpoint rather than an ordinary CRUD endpoint. (*Include what you would do about the amount object — value, currency, precision — and about the limit and insufficient-funds paths.*)

3.2 What is the Idempotency-Key for, how would you test that it works, and what should happen when a client retries after a network timeout or after receiving a 503?

3.3 How would you verify that the money actually moved — beyond checking that the API returned 201?

Task 4 — Automation Design (Design & Thinking Focus)

Suggested time: 35 minutes

Objective

This task assesses **automation design thinking, not coding ability**.

- Do **not** implement the automation.
- Do **not** submit a working framework.
- Do explain how you would design a scalable, maintainable solution, and why.

Pseudocode or a short illustrative snippet is welcome where it makes your reasoning clearer. A full implementation is not, and will not earn extra credit.

Scenario

You must automate regression coverage for the **Bank Operations Console** — the internal screen where bank staff configure onboarding rules, KYC thresholds, transfer limits, fees, notifications, channels and audit settings.

An HTML reference page is attached (`Bank_Admin_Console_Demo.html`). **Open it to understand the UI behaviour**. Do not automate against it — it is a visual reference only.

The technical challenge

Tabs

- The console has multiple tabs rendered as headers.
- Switching tabs or scrolling may update or replace field elements, and dynamically generated identifiers can change during a session.

Fields inside each tab

Each tab contains text fields, checkboxes and drop-down lists.

- Every configurable item follows the **same stable parent-wrapper structure**. The wrapper can be located by its **visible label text** or by a **unique descriptive attribute**.
- The **child control** inside that wrapper may be a text input, a checkbox or a dropdown.
- The child control carries a **dynamically generated identifier** — for example `slot="field-145"` — whose value changes during the session. It must not be used as your primary locator.

Required scenario

Design an automated flow that:

- Updates **at least 12 fields**
- Across **at least 4 different tabs**
- Saves the configuration

- Verifies the values persisted after a page reload

Note: the reference page has no Save button and does not persist anything. Saving and reload persistence exist in the production system and are deliberately not implemented in the reference page — design for them as described above.

What we would like from you

Cover these six areas:

1. **Project structure and Page Object Model design** — folders, layers, naming (show the structure as a tree). What is a page object here, how do tabs and repeated field types fit, and how do you avoid one 900-line class?
2. **Locator strategy** — how you identify a field reliably given the dynamic identifier, and how you handle the three different control types behind one interface. Be specific.
3. **Handling dynamic DOM** — elements changing on tab switch and scroll, stale references, waiting strategy. What do you *not* do?
4. **Test data strategy** — no hard-coded values. Where data lives, how you handle values that must be unique per run, and how you leave the environment clean. Assume test environments are **shared with other teams, data is restricted, and you cannot reset the database on demand** — tell us how that shapes your design.
5. **Reusability and scalability** — what a new engineer writes to add a 5th tab or a 20th field; execution time, parallelism, CI integration and reporting as the suite grows from 7 tabs to 20.
6. **Tooling** — the stack you would choose and **why**. Tell us the trade-off you accepted, not only the choice you made.

Task 5 — Quality Strategy & Release Judgement

Required for Lead candidates. Optional bonus for Senior candidates. Mid-Senior candidates may skip this task.

Suggested time: 30 minutes. Maximum one and a half pages.

Scenario

You are the QA lead on a delivery team building a new **international transfer** journey for a bank client. Go-live is in **3 working days**. The release has been announced to the client's business stakeholders and to their marketing team.

This is the current state:

- The regression suite passes at **68%**. Around 30 of the failures are believed to be “just flaky” but nobody has investigated them this sprint.
- **Two open critical defects** in the onboarding/KYC area, which is not part of this release but shares the customer profile service.
- **UAT is not signed off**. The client's business team has run about half of their scripts.
- The FX rate integration was deployed to the staging environment **yesterday** and has had one round of happy-path testing.
- Two of your four QA engineers are new to the project, having joined 5 weeks ago.
- Your delivery manager has asked you, in writing, to confirm that “QA is green”.

What we would like from you

A short memo — **maximum one and a half pages** — covering:

5.1 Your **go / no-go recommendation** and the reasoning behind it. If your answer is conditional, state the conditions precisely.

5.2 What you would do in the next 72 hours, in priority order, and what you would drop.

5.3 How you would communicate this to (a) your delivery manager and (b) the client. Note where those two messages differ and where they must not.

5.4 What you would change in how the team works so that you are not in this position again next release, and two or three metrics you would put in place to make quality visible to the client. Be concrete — name the practice, not the principle — and say what you would *not* measure.

How We Will Assess Your Submission

We score against these areas. They are weighted differently by level, but all of them matter.

Area	What we are looking for
Requirement analysis & QA mindset	Did you find the gaps? Did you ask the right questions instead of assuming?
Risk-based thinking	Do you prioritise by business impact, or test everything equally?
Test design & coverage	Are your cases meaningful, well-structured, and free of duplication?
Defect reporting	Could a developer reproduce and fix from your report without asking you anything?
API & integration understanding	Do you understand what happens beneath the UI?
Automation design	Is your design maintainable, scalable, and honest about trade-offs?
Domain awareness	Do you reason like someone testing money movement, not a generic web form?
Communication	Is your submission clear, structured and readable?

What happens next

After you submit, two of our technical leads will review your work and meet with you for a technical discussion. A good part of that session will be a conversation about *your* submission — the decisions you made, the alternatives you considered, and how you would extend it. Come ready to defend and to reconsider your choices; both are welcome.

We are genuinely interested in your thinking. Please treat this as a conversation starter rather than an exam paper.

Good luck, and thank you for the time you are giving us.